

Chapitre 1 : Syntaxe, Variables et Types de Données

1.1. Philosophie et Syntaxe de Base

Python se distingue par sa lisibilité. Contrairement à de nombreux langages (C++, Java, JS) qui utilisent des accolades `{}` et des points-virgules `;`, Python utilise l'**indentation** (4 espaces) pour définir les blocs de code.

```
# Un commentaire sur une ligne

"""
Un commentaire multi-lignes
ou une docstring de documentation.
"""

if True:
    print("Ce bloc est indenté")
```

1.2. Typage Dynamique et Fort

- **Dynamique** : Pas besoin de déclarer le type lors de la création d'une variable.
- **Fort** : Python n'effectue pas de conversions de types implicites incompatibles (par exemple, additionner une chaîne et un nombre lève une erreur `TypeError`).

```
x = 10          # x est un int
x = "Dix"       # Réaffectation possible vers un str

# Typage fort :
# print("Age : " + 25) # TypeError!
print("Age : " + str(25)) # Conversion explicite requise
```

1.3. Types Primitifs

Type	Description	Exemple
<code>int</code>	Entier de taille arbitraire	<code>42</code> , <code>-1000</code> , <code>1_000_000</code>
<code>float</code>	Nombre à virgule flottante (64-bit)	<code>3.14</code> , <code>-0.001</code> , <code>2e-3</code>
<code>str</code>	Chaîne de caractères (Unicode par défaut)	<code>"Python"</code> , <code>'Code'</code>
<code>bool</code>	Booléen (<code>True</code> ou <code>False</code>)	<code>True</code> , <code>False</code>
<code>NoneType</code>	Représente l'absence de valeur	<code>None</code>

1.4. Type Hinting (Annotations de Type)

Introduit avec la **PEP 484**, le type hinting permet d'indiquer le type attendu sans forcer le typage à l'exécution :

```
nom: str = "Alice"
age: int = 30
taille: float = 1.75
est_actif: bool = True
```

1.5. Formatage de Chaînes : f-strings

Les **f-strings** (depuis Python 3.6) sont la méthode recommandée pour formater des chaînes :

```
prix = 49.99
quantite = 3

# Inclusion d'expressions directement dans la chaîne
message = f"Total : {prix * quantite:.2f} €" # Output: Total : 149.97 €
```

Chapitre 2 : Structures de Contrôle et Fonctions

2.1. Conditions (**if**, **elif**, **else**)

Les conditions évaluent la valeur truthy/falsy des expressions.

Valeurs considérées comme *False* : **None**, **0**, **0.0**, **""** (chaîne vide), **[]** (liste vide), **{}** (dict vide), **set()**.

```
age = 20

if age < 12:
    print("Enfant")
elif age < 18:
    print("Adolescent")
else:
    print("Adulte")

# Expression conditionnelle (Ternaire)
statut = "Majeur" if age >= 18 else "Mineur"
```

2.2. Boucles (**for** et **while**)

****La boucle **for******

S'utilise pour parcourir n'importe quel itérable (liste, chaîne, `range`, etc.).

```
# Utilisation de range(debut, fin, pas)
for i in range(1, 10, 2):
    print(i) # 1, 3, 5, 7, 9

# Parcours avec enfilage d'index via enumerate()
fruits = ["pomme", "banane", "cerise"]
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
```

****La boucle `while` et la clause `else`****

La clause `else` d'une boucle s'exécute uniquement si la boucle s'est terminée normalement (sans `break`).

```
compteur = 0
while compteur < 3:
    print(compteur)
    compteur += 1
else:
    print("Boucle terminée sans interruption")
```

2.3. Fonctions et Signature Avancée

```
def creer_utilisateur(
    nom: str,
    email: str,
    role: str = "Utilisateur",
    *droits: str,
    **metadonnees: str
) -> dict:
    """
    *droits capture les arguments positionnels supplémentaires sous forme
    de tuple.
    **metadonnees capture les arguments nommés supplémentaires sous forme
    de dictionnaire.
    """
    return {
        "nom": nom,
        "email": email,
        "role": role,
        "droits": droits,
        "metadonnees": metadonnees
    }
```

```
user = creer_utilisateur(  
    "Jean",  
    "jean@example.com",  
    "Admin",  
    "READ", "WRITE",  
    connexion="2026-03-30", ip="127.0.0.1"  
)
```

Chapitre 3 : Structures de Données Intégrées

3.1. Listes (**list**)

Projections ordonnées, mutables et dynamiques.

```
# Modifiables à la volée  
nbres = [1, 2, 3]  
nbres.append(4)      # [1, 2, 3, 4]  
nbres.extend([5, 6]) # [1, 2, 3, 4, 5, 6]  
nbres.pop()          # Supprime et renvoie le dernier élément (6)  
  
# Slicing : [debut:fin:pas]  
sous_ensemble = nbres[1:4] # Éléments de l'index 1 à 3  
inverse = nbres[::-1]      # Inverser la liste
```

3.2. Tuples (**tuple**)

Séquences ordonnées mais **immutables**. Une fois créés, leurs éléments ne peuvent plus être modifiés, ajoutés ou supprimés.

```
point = (10, 20)  
# point[0] = 15 # TypeError: 'tuple' object does not support item  
# assignment  
  
# Unpacking (Désencapsulation)  
x, y = point
```

3.3. Dictionnaires (**dict**)

Structures clé-valeur basées sur une table de hachage. Complexité de recherche moyenne : $\mathcal{O}(1)$.

```
configs = {"hote": "localhost", "port": 8080}
```

```
# Accès sécurisé avec valeur par défaut
port = configs.get("port", 80)
db = configs.get("database", "postgres")

# Méthodes principales
cles = configs.keys()
valeurs = configs.values()
items = configs.items() # Tuple (clé, valeur)
```

3.4. Ensembles (set)

Collections non ordonnées d'éléments **uniques**. Utiles pour supprimer les doublons et effectuer des opérations mathématiques sur des ensembles.

```
set_a = {1, 2, 3, 4}
set_b = {3, 4, 5, 6}

# Opérations d'ensemble
union = set_a | set_b      # {1, 2, 3, 4, 5, 6}
intersection = set_a & set_b # {3, 4}
difference = set_a - set_b  # {1, 2}
```

3.5. Compréhensions de Séquences

Syntaxes concises et optimisées pour construire de nouvelles structures à partir d'itérables.

```
# Compréhension de liste
carres = [x**2 for x in range(10) if x % 2 == 0]
# Résultat : [0, 4, 16, 36, 64]

# Compréhension de dictionnaire
mots = ["python", "code", "dev"]
longueurs = {mot: len(mot) for mot in mots}
# Résultat : {'python': 6, 'code': 4, 'dev': 3}

# Compréhension d'ensemble
chiffres = {int(chiffre) for chiffre in "11223344"}
# Résultat : {1, 2, 3, 4}
```